

Python Bridge – Addon-Dokumentation

Version 0.1.0 | Godot 4.x + Python 3.8+ | WebSocket-basierte Python-Prozess-Instanzen

Inhalt

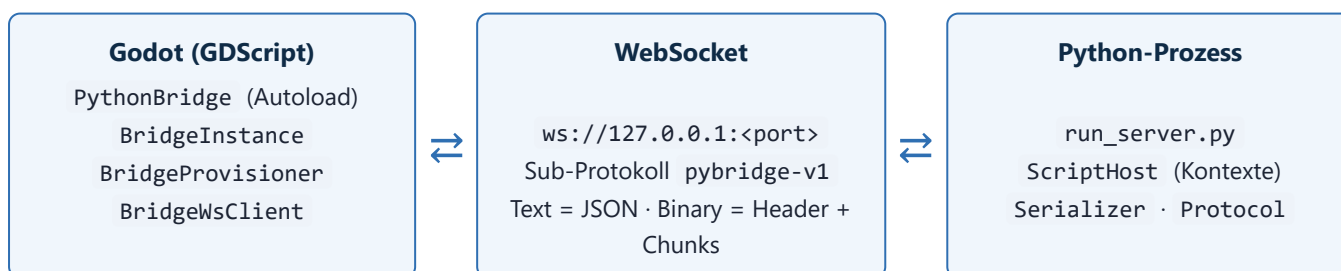
- 1. Übersicht
- 2. Architektur
- 3. Installation in einem neuen Projekt
- 4. GDScript-API-Referenz
- 5. Datentransport & Typen
- 6. Python-Seite
- 7. Grundgerüst / Programmstruktur (Schema)
- 8. Lebenszyklus & Provisioning
- 9. Fehlerbehandlung & Troubleshooting

1. Übersicht

Die **Python Bridge** startet aus Godot heraus einen oder mehrere **Python-Subprozesse** (je Instanz ein eigener Prozess) und kommuniziert mit ihnen über **WebSocket** auf `127.0.0.1`. Sie übernimmt automatisch:

- **venv-Verwaltung**: legt pro Projekt ein virtuelles Environment unter `python_bridge/venv/` an (einmalig).
- **Dependency-Management**: installiert `websockets` + frei konfigurierbare Pakete (z. B. `numpy`, `scipy`) per `pip`.
- **Python-Erkennung**: findet eine installierte Python (3.8+) selbst, ohne fest verdrahteten Pfad.
- **Typisierten Transport**: JSON für Skalare, Binär-Chunks für große Daten (NumPy-Arrays, Bytes, Bilder).
- **Persistente Skript-Kontexte**: Skripte und deren Zustand (Importe, Variablen, Funktionen) bleiben in der Instanz erhalten.

2. Architektur



Der Ablauf: Godot startet den Python-Server als Subprozess. Der Server schreibt Port + PID in `python_bridge/tmp/<instanz>.json`. Godot liest die Datei, verbindet sich per WebSocket, sendet ein `hello` und gilt nach der `hello_ack`-Antwort als **READY**. Jede Anfrage trägt eine eindeutige `id`; Antworten werden darüber zugeordnet.

Nachrichten-Typen (Protokoll v1): `hello` → `hello_ack` · `execute` → `response` · `ping` → `pong` · `shutdown` · `event` · `status`

3. Installation in einem neuen Projekt

1. Den Ordner `addons/python_bridge` (aus `python_bridge_addon.zip`) in das Zielprojekt nach `addons/` entpacken.
2. Godot öffnen → **Projekt** → **Projekteinstellungen** → **Plugins** → „Python Bridge“ aktivieren. Das Plugin registriert den Autoload-Singleton `PythonBridge` automatisch.
3. Python 3.8+ muss installiert sein (die Bridge findet sie über `PATH`, `PYTHON_PATH` oder `python_executable`).
4. Beim ersten Start wird automatisch `python_bridge/venv/` erzeugt und `websockets` (+ konfigurierte Pakete) installiert – das kann 1–3 Minuten dauern.

Hinweis: Das venv liegt im Projektordner. Damit Godot es nicht als Ressourcen importiert, enthält `python_bridge/venv/` eine `.gdignore`-Datei. Bei Projekten mit einem bestehenden venv muss diese Datei vorhanden sein.

4. GDScript-API-Referenz

Alle Funktionen sind **Instanz-Methoden** des Autoload-Singletons `PythonBridge` und dürfen daher `await` verwenden. Aufrufe, die Python ausführen, liefern ein `PythonBridgeResult` (siehe Abschnitt 4.3).

4.1 PythonBridge (Autoload)

Funktion	Beschreibung
<code>configure(cfg: Dictionary)</code>	Globale Einstellungen setzen. Schlüssel: <code>autostart</code> (bool, Standard <code>false</code>), <code>workspace_dir</code> (res://-Pfad, Standard <code>res://python_bridge</code>), <code>python_executable</code> (String, leer = automatische Suche), <code>dependencies</code> (Array/PackedStringArray, z. B. <code>["numpy"]</code>).
<code>start_instance(name := "default")</code>	Startet (bzw. holt) eine Instanz und wartet bis READY (Timeout 180 s). Liefert <code>PythonBridgeResult</code> .
<code>get_instance(name := "default")</code>	Liefert die <code>BridgeInstance</code> (oder <code>null</code>), ohne sie zu starten.
<code>instance_status(name := "default")</code>	Status-String der Instanz (<code>"none"</code> , <code>"provisioning"</code> , <code>"starting"</code> , <code>"connecting"</code> , <code>"handshake"</code> , <code>"ready"</code> , <code>"error"</code> , <code>"stopped"</code>).
<code>execute(code, input := {}, name := "default", timeout := 30)</code>	Führt temporären Python-Code in einem Wegwerf-Kontext aus. Das Skript liest <code>input</code> und schreibt das Ergebnis in <code>result</code> .
<code>create_script(script_id, code, subfolder := "")</code>	Schreibt ein dauerhaftes Skript nach <code>res://python_bridge/scripts/<subfolder>/<id>.py</code> . Wiederverwendbar (überschreibt). Liefert <code>PythonBridgeResult</code> .
<code>get_script_source(script_id)</code>	Gibt den Quelltext des Skripts als String zurück (leer, wenn nicht vorhanden).
<code>execute_script(script_id, input := {}, name := "default", timeout := 30)</code>	Führt ein dauerhaftes Skript aus (Konvention: <code>input</code> / <code>result</code>).
<code>call_script(script_id, function, args := [], kwargs := {}, name := "default", timeout := 30)</code>	Ruft eine benannte Funktion im Skript-Kontext auf; der Rückgabewert der Funktion ist das Ergebnis.
<code>script_path_for(script_id, subfolder := "")</code>	res://-Pfad eines Skripts.
<code>resolve_script_path(script_id)</code>	Erlaubt volle Pfade <i>oder</i> <code>id</code> bzw. <code>unterordner/id</code> innerhalb von <code>scripts/</code> .
<code>stop_instance(name := "default")</code> · <code>stop_all()</code> · <code>shutdown()</code>	Beendet Instanz(en) inkl. Python-Prozess (sendet <code>shutdown</code> , killt danach den Prozess).
<code>poll()</code>	Treibt alle Instanzen pro Frame (wird automatisch in <code>_process</code> und vom Editor-Plugin aufgerufen).
<code>system()</code>	Statischer Helfer: findet den Autoload-Node über den Szenenbaum.

Signale: `pulse` (jeden Frame), `instance_state_changed(instance, state)`.

4.2 BridgeInstance

Eine Instanz = **ein Python-Subprozess + ein WebSocket-Kanal**. Meist reicht die `PythonBridge`-Facade; für Spezialfälle (eigener Timeout, Status-Logging) kann man die Instanz direkt nutzen.

Funktion	Beschreibung
<code>run(context_id, source, input, command := "run", function := "", args := [], kwargs := {}, timeout := 30)</code>	Unterste Aufruf-Ebene: Code im benannten Kontext ausführen ("run") oder Funktion rufen ("call").
<code>wait_ready(timeout := 180)</code>	Wartet (via <code>await</code> auf <code>pulse</code>) bis READY; liefert <code>bool</code> .
<code>status_text()</code>	Aktueller Zustand als String (siehe 4.1).
<code>is_active()</code> · <code>is_ready_immediately()</code>	Zustandsprüfungen.
<code>last_error_message()</code>	Letzter Fehlertext (z. B. Provisioning-/Verbindungsfehler).
<code>next_context_id()</code>	Liefert eine neue, eindeutige Kontext-ID (für temporäre Kontexte).
<code>stop()</code> · <code>shutdown_now()</code>	Beendet die Instanz (graceful bzw. sofort).

Signale: `instance_ready` , `state_changed(instance, state)` . **Zustände:** NONE → PROVISIONING → STARTING → CONNECTING → HANDSHAKE → READY (bzw. ERROR/STOPPED).

4.3 PythonBridgeResult

Feld	Beschreibung
<code>ok</code> / <code>is_ok()</code> / <code>is_error()</code>	Erfolg? Fehler?
<code>status</code>	"ok" , "error" (Python-Exception), "timeout" , "not_ready" , "down" (Prozess getrennt), "internal" (Tool-Problem).
<code>value</code>	Ergebnis bei Erfolg (typisiert, siehe Abschnitt 5).
<code>error</code>	Dictionary { <code>type</code> , <code>message</code> , <code>traceback</code> } bei Fehlern.
<code>error_message()</code>	Kurztext des Fehlers (message oder status).
<code>request_id</code> · <code>meta</code>	Interne Request-ID · Meta-Daten (z. B. <code>duration_ms</code>).

5. Datentransport & Typen

Skalare bleiben lesbares JSON. Strukturierte Werte werden als getaggte Objekte `{"$pb": "<tag>", ...}` kodiert. Große Binär-Blobs (> 512 Bytes) wandern nicht als Base64 in den JSON-Header, sondern als **Binär-Chunks** im Frame (`U32LE(Header-Länge) + HeaderJSON + [U32LE(Chunk-Länge) + Chunk]*`).

Python → Godot	Godot-Typ
<code>None</code> / <code>bool</code> / <code>int</code> / <code>float</code> / <code>str</code>	<code>null</code> / <code>bool</code> / <code>int</code> / <code>float</code> / <code>String</code>
<code>list</code> / <code>tuple</code> / <code>set</code>	<code>Array</code>

<code>dict</code>	Dictionary (Schlüssel müssen str/int/float/bool sein)
<code>bytes</code> / <code>bytearray</code>	<code>PackedByteArray</code>
<code>numpy.ndarray</code> (float64)	<code>PackedFloat64Array</code> (1D) bzw. <code>Array</code> von Zeilen (>1D)
<code>numpy.ndarray</code> (float32/int32/int64/...)	<code>PackedFloat32Array</code> / <code>PackedInt32Array</code> / <code>PackedInt64Array</code> / ... (je nach dtype)
<code>numpy.integer</code> / <code>numpy.floating</code>	<code>int</code> / <code>float</code>
beliebiges anderes Objekt	<code>String</code> (repr-Fallback, Tag <code>pyobject</code>)

Godot → Python	Python-Typ
<code>Vector2/3/4</code> , <code>Color</code> , <code>Transform3D</code>	<code>list</code> der Komponenten (z. B. <code>[x, y, z]</code>)
<code>PackedByteArray</code> (groß)	<code>bytes</code> (als Chunk)
<code>PackedInt/Float</code> -Arrays	<code>bytes</code> (roh, für <code>numpy.frombuffer</code>)
<code>Array</code> / <code>Dictionary</code>	<code>list</code> / <code>dict</code>
<code>Image</code>	rohe Pixel-Daten + Metadaten (w, h, format)

Wichtig (Endianness): Längen-Präfixe im Binärformat sind **Little-Endian** auf beiden Seiten (`struct.pack("<I", ...)` in Python, `decode_u32` /explizite Byte-Kodierung in Godot). Beide Seiten müssen dieselbe Variante verwenden.

6. Python-Seite

6.1 Skript-Konventionen

- `execute` / `execute_script` : Das Skript erhält die Eingabe in der Variablen `input` und legt das Ergebnis in `result` ab.
- `call_script` : Die benannte Funktion wird mit `args` / `kwargs` aufgerufen; ihr **Rückgabewert** wird an Godot übergeben.
- **Persistente Kontexte:** Jede `context_id` hat einen eigenen Namespace (`ScriptContext`). Importe, definierte Funktionen und Variablen bleiben zwischen Aufrufen desselben Kontexts erhalten. Temporäre Ausführungen (`execute`) verwenden Wegwerf-IDs.
- **Fehler:** Python-Exceptions werden strukturiert gemeldet (`{"type", "message", "traceback"}`) – der Server-Prozess bleibt am Leben.

6.2 Aufbau des Python-Pakets

Datei	Zweck
<code>run_server.py</code>	Entry-Point: CLI-Parameter (<code>--bind --port --tmpdir --tag</code>), startet den <code>asyncio</code> -Server.
<code>python_bridge/server.py</code>	WebSocket-Server: Routet <code>hello</code> , <code>ping</code> , <code>execute</code> , <code>shutdown</code> ; schreibt Port+PID in die Tmp-Datei.
<code>python_bridge/executor.py</code>	<code>ScriptHost</code> : verwaltet Kontexte, führt <code>run</code> / <code>call</code> aus, fängt Exceptions.

<code>python_bridge/protocol.py</code>	Frame-Bau/Parsing: <code>build_text</code> , <code>build_binary</code> , <code>parse</code> (Text + Binary, Little-Endian).
<code>python_bridge/serializer.py</code>	<code>encode_obj</code> / <code>decode_obj</code> : typisierte Serialisierung inkl. NumPy-Arrays und Chunks.
<code>requirements.txt</code>	Minimale Abhängigkeit: <code>websockets>=11</code> (weitere Pakete pro Projekt konfigurierbar).

7. Grundgerüst / Programmstruktur (Schema)

Typische Architektur eines Godot-Skripts mit zugehöriger Python-Datei – ohne konkrete Inhaltslogik. Das GDScript ist der **Controller** (Lifecycle, UI, Rendering), die Python-Datei der **Rechner** (Logik, Daten, Wissenschaft/ML). Beide sind über `SCRIPT_ID` verbunden.

7.1 GDScript (z. B. `node_3d.gd`) – Controller

```
# =====
# Struktur eines Godot-Skripts, das die Python-Bridge nutzt
# =====
extends Node3D          # je nach Zweck: Node, Node2D, Control, ...

# --- 1) Konstanten -----
const SCRIPT_ID  := "mein_skript"    # Name der Python-Datei (ohne .py)
const PYTHON_CODE := ""
# hier steht der Python-Quellcode; wird beim ersten Start nach
# res://python_bridge/scripts/mein_skript.py geschrieben
"""

# --- 2) Variablen -----
var _bereit := false

# --- 3) Lifecycle -----
func _ready() -> void:
    await _python_starten()

func _exit_tree() -> void:
    PythonBridge.stop_all()          # Python-Prozesse sauber beenden

# --- 4) Python-Bridge initialisieren -----
func _python_starten() -> void:
    # optional: Pakete konfigurieren, z. B. ["numpy"]
    PythonBridge.configure({ "dependencies": [] })

    var start := await PythonBridge.start_instance("default")
    if start.is_error():
        push_error("Python-Start: " + start.error_message())
        return

    # Skript nur anlegen, wenn es noch nicht existiert
    if PythonBridge.get_script_source(SCRIPT_ID) == "":
        PythonBridge.create_script(SCRIPT_ID, PYTHON_CODE)

    _bereit = true

# --- 5) Python-Aufrufe (Funktion rufen / Skript ausführen) -----
func _python_aufrufen(arg1: Variant, arg2: Variant) -> void:
    if not _bereit: return

    var result := await PythonBridge.call_script(
        SCRIPT_ID, "meine_funktion", [arg1, arg2])

    if result.is_error():
        push_error(result.error_message())    # Python-Fehler (type/message/traceback)
        return

    var daten: Variant = result.value        # typisiertes Ergebnis, s. Abschnitt 5
    # ... Ergebnis weiterverarbeiten (z. B. Mesh/Points aufbauen) ...

# --- 6) Weitere Helfer -----
# _process(_delta): Frames/Animation, Rendering, Eingabe ...
```

7.2 Python-Datei (`res://python_bridge/scripts/mein_skript.py`) – Rechner

```
# =====
# PythonBridge-Skript: mein_skript
# Konventionen:
#   execute/execute_script : Eingabe in `input`, Ergebnis in `result`
#   call_script            : Rückgabewert der Funktion ist das Ergebnis
#   Kontext bleibt zwischen Aufrufen erhalten (Importe, Variablen)
# =====
import numpy as np      # optional - nur wenn als Dependency konfiguriert

# --- Funktionen (per call_script aufrufbar) -----
def meine_funktion(a, b):
    # ... deine Logik (Berechnungen, Datenaufbereitung) ...
    ergebnis = None
    return ergebnis      # geht direkt an Godot (result.value)

# --- Skriptausführung (per execute_script) -----
# input  = Eingabe aus Godot (z. B. dict, Array, ndarray)
# result = Ausgabe nach Godot
result = None
```

7.3 Typischer Aufruf-Ablauf

```
configure() → await start_instance() → create_script() (einmalig) → await call_script() /
await execute_script() → Ergebnis in result.value → ... → stop_all() (beim Beenden).
```

8. Lebenszyklus & Provisioning

1. **PROVISIONING** – Workspace-Ordner anlegen (`scripts/` , `tmp/` , `config/`), Python erkennen (≥ 3.8), Bridge-Python nach `python_bridge/bridge/` kopieren.
2. **venv** – falls nicht vorhanden: `python -m venv ../venv` (nicht-blockierend, Timeout 120 s).
3. **pip** – `pip install websockets>=11` + konfigurierte Dependencies (Timeout 300 s, Log in `tmp/pip.log`).
4. **STARTING** – `run_server.py` als Subprozess starten; Server schreibt `tmp/<instanz>.json` (Port + PID).
5. **CONNECTING** → **HANDSHAKE** – WebSocket-Verbindung aufbauen, `hello` senden.
6. **READY** – `hello_ack` empfangen; Requests sind jetzt möglich.

Bei Verbindungsabbruch versucht die Instanz bis zu **2 Reconnects** (`MAX_RECONNECTS`); offene Requests werden mit Status `"down"` aufgelöst. Prozess-Ende wird über den WebSocket-Zustand erkannt (nicht über Exit-Codes), die PID dient nur dem gezielten `kill()`.

9. Fehlerbehandlung & Troubleshooting

Problem / Status	Ursache & Lösung
<code>status = "not_ready"</code>	Instanz wurde nicht gestartet oder noch nicht READY – vor Aufrufen <code>await start_instance()</code> nutzen bzw. <code>wait_ready()</code> .

<code>status = "timeout"</code>	Antwort kam nicht in <code>timeout_sec</code> (Standard 30 s). Timeout erhöhen oder Python-Code beschleunigen.
<code>status = "error"</code>	Python-Exception im Nutzer-Code – Details in <code>result.error</code> (<code>type</code> / <code>message</code> / <code>traceback</code>).
<code>status = "down"</code>	Python-Prozess getrennt (WebSocket geschlossen) – z. B. Absturz im Server oder <code>stop()</code> . Prozess-Log prüfen.
<code>status = "internal"</code>	Tool-Infrastrukturfehler (z. B. Skriptdatei nicht schreibbar, Sende-/Parsefehler).
„Kein Python gefunden“	<code>python_executable</code> setzen, <code>PYTHON_PATH</code> definieren oder Python zum <code>PATH</code> hinzufügen.
venv/pip hängt	Timeout 120/300 s; Details in <code>python_bridge/tmp/pip.log</code> . ggf. venv-Ordner löschen und neu provisionieren lassen.
„Python-Prozess getrennt“ beim ersten Aufruf	Meist ein Server-/Protokollproblem beim Binär-Versand – Version von <code>addons/python_bridge/python</code> (Quelle) und <code>python_bridge/bridge</code> (Laufzeit-Kopie) muss identisch sein; das Plugin kopiert bei jedem Start automatisch.
Godot importiert venv-Dateien	<code>python_bridge/venv/.gdignore</code> muss existieren (in der Zip enthalten).
Werte kommen als <code>String</code> an	Objekt war nicht serialisierbar → <code>pyobject</code> -Fallback (<code>repr</code>). Nur JSON-fähige Typen, <code>bytes</code> , <code>ndarray</code> , Listen/Dicts verwenden.

Python Bridge v0.1.0 – dokumentiert nach dem Quellcode in `addons/python_bridge/` (Godot-Seite: `core/` , Python-Seite: `python/`).